INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2025/2026

Tanggal / Sesi	:	Rabu, 17 September 2025
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	04/ 02
Aktivitas	:	Classes and Objects, Inheritance
Durasi	:	110 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Rabu, 17 September 2025 Pukul 23.59
Tujuan	:	- Mahasiswa mampu membuat class dan menginstasiasi objek dari sebuah kelas dan inheritance. - Melatih *computational thinking*: memformulasikan masalah → merancang solusi → menulis kode → memverifikasi output

Class dan Object

Dalam java, Class dan Object merupakan konsep dasar Pemograman Berorientasi Objek (OOP) yang digunakan untuk merepresentasikan konsep dan entitas dunia nyata.

Class

Sebuah **Class** di java merupakan sebuah template (*blueprint*) yang digunakan untuk membuat entitas benda nyata yang disebut Object, yang memiliki karakteristik dan properti yang sama. Mari kita lihat bagaimana penggunaan dasar dari Class di java. Kita dapat melihat pada code di bawah.

```
class Student {
    int id;
    String n;

    public Student(int id, String n) {
        this.id = id;
        this.n = n;
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student(10, "Alice");

        System.out.println(s1.id);
        System.out.println(s1.n);
    }
}
```

Output:

```
run:
10
Alice
```

```
access_modifier class<class_name> { //merupakan deklarasi class di java
    data member;
    method;
    constructor;
    nested class;
    interface
    ;
}
```

Object

Objek adalah instance dari sebuah class yang dibuat untuk menggunakan atribut dan method dari class tersebut. Sebuah program Java biasanya membuat banyak objek yang saling berinteraksi dengan memanggil method. Sebuah objek terdiri dari *State* (attributes), *Behavior* (methods) dan *Identity* (nama unik pada sebuah objek).

Inisialisasi Objek

Operator `new` membuat instance Class dengan mengalokasikan memori untuk objek baru dan mengembalikan referensi ke memori ke memori tersebut. Operator `new` juga memanggil konstruktor Class. Kita dapat melihat contoh code berikut

```
public class Dog {
    String name;
    String breed;
    int age;
    String color;

    // Constructor
    public Dog(String name, String breed, int age,
               String color)
    {
        this.name = name;
        this.breed = breed;
        this.age = age;
        this.color = color;
    }

    // method 1
    public String getName() {
      return name;
    }

    // method 2
    public String getBreed() {
      return breed;
    }

    // method 3
    public int getAge() {
      return age;
    }
```

```
    // method 4
    public String getColor() {
      return color;
    }

    @Override public String toString()
    {
        return ("Name is: " + this.getName()
                + "\nBreed, age, and color are: "
                + this.getBreed() + "," + this.getAge()
                + "," + this.getColor());
    }

    public static void main(String[] args)
    {
        Dog tuffy
            = new Dog("tuffy", "papillon", 5, "white");
        System.out.println(tuffy.toString());
    }
}
```

Output:

```
run:
Name is: tuffy
Breed, age, and color are: papillon,5,white
```

Inisialisasi Objek Menggunakan Method/Function

Sebuah objek di Java juga dapat diinisialisasi melalui method atau function khusus. Caranya adalah dengan terlebih dahulu membuat objek menggunakan operator new, kemudian memanggil method yang bertugas mengatur nilai atribut atau variabel instance dari objek tersebut. Metode ini sering disebut initializer method atau setter method.

```java
public class Geeks {

    static String name;
    static float price;

    static void set(String n, float p) {
        name = n;
        price = p;
    }

    static void get()
    {
        System.out.println("Software name is: " + name);
        System.out.println("Software price is: " + price);
    }

    public static void main(String args[])
    {
        Geeks.set("Visual studio", 0.0f);
        Geeks.get();
    }
}
```

Output:

```
run:
Software name is: Visual studio
Software price is: 0.0
```

Cara Membuat Objek dari Sebuah Class

Ada 4 cara untuk membuat objek di java. Meskipun kata kunci new merupakan cara uatama untuk membuat objek, metode lain juga secara internal bergantung pada kata kunci new untuk membuat instans.

1. **Menggunakan new Keyboard**

   Ini merupakan cara paling umum dan umum untuk membuat objek di class.

   ```
   // membuat object
   Test t = new Test();
   ```

2. **Menggunakan Reflection**

   Reflection merupakan fitur hebat di java yang memungkin program untuk memeriksa dan mengubah struktur serta perilakunya sendiri saat runtime.

```
class Student { //Asumsikan kelas Student sudah ada (tunjukkan definisinya)
    public Student() {}
}

public class Main {
    public static void main(String[] args) {
        try {
            Class<?> c = Class.forName("Student");
            Student s2 = (Student) c.getDeclaredConstructor().newInstance();
            System.out.println("Object created: " + s2);
        } catch (ClassNotFoundException e) {
            System.err.println("Class not found!");
        } catch (NoSuchMethodException e) {
            System.err.println("No default constructor!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3. Menggunakan clone() method

Metode `clone()` terdapat di dalam class objek. Metode ini membuat dan mengembalikan salinan objek.

```
class Geeks implements Cloneable {
    @Override //method 1
    protected Object clone()
        throws CloneNotSupportedException
    {
        return super.clone();// Super() mengacu ke parent class
    }
    String name = "GeeksForGeeks";

    public static void main(String[] args)// Method 2
    {
        Geeks o1 = new Geeks();

        try {
            Geeks o2 = (Geeks)o1.clone();
            System.out.println(o2.name);
        }
        catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
run:
GeeksForGeeks
```

4. Deserialization

Deserialisasi merupakan teknik membaca objek dari status tersimpan dalam sebuah class.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class Main {
    public static void main(String[] args)
    {
        // Serialization
        try (ObjectOutputStream out
             = new ObjectOutputStream(
                 new FileOutputStream("student.ser"))) {
            out.writeObject(new Student("Alice"));
        }
        catch (IOException e) {
            e.printStackTrace();
        }
        // Deserialization
        try (ObjectInputStream in = new ObjectInputStream(
                 new FileInputStream("student.ser"))) {
            Student s = (Student)in.readObject();
            System.out.println(s);
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
run:
Student: Alice
```

Membuat Beberapa Objek dengan Satu Jenis

Secara real-time, kita membutuhkan objek yang berbeda dari suatu class dengan metode yang berbeda pula. Membuat sejumlah referensi untuk menyimpannya bukanlah praktik yang baik, oleh karena itu kita mendeklarasikan variabel referensi statis dan menggunakannya kapan pun diperlukan. Dalam sistem pewarisan, kita menggunakan variabel referensi kelas induk untuk menyimpan objek subclass.

```
class Animal {
}
class Dog extends Animal {
}
class Cat extends Animal {
}
public class Test {
    Animal obj = new Dog();  // Menggunakan Dog object
    obj = new Cat();  // Menggunakan Cat object
}
```

Membuat Objek Anonymous

Objek anonym adalah objek yang diinstansiasi tanpa menyimpan referensinya dalam variabel. Objek ini digunakan untuk operasi satu kali (mis, pemanggilan metode) dan langsung dibuang setelah digunakan.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class Main extends Application {
    public static void main(String[] args) { launch(args); }
    @Override public void start(Stage stage)
    {
        Button button = new Button("Click Me");
        button.setOnAction(
            event
            -> System.out.println(
                "Button clicked!")); // Anonymous object melalui lambda

        StackPane root = new StackPane(button);
        stage.setScene(new Scene(root, 300, 200));
        stage.show();
    }
}
```

Class Object di Java

Dalam pemrograman berorientasi objek (OOP) di Java, Class **`Object`** merupakan class induk (superclass) dari semua class yang ada. Artinya, setiap class yang kita buat di Java secara langsung atau tidak langsung mewarisi class `Object`, meskipun kita tidak menuliskan `extends Object` secara eksplisit. Class `Object` berada di dalam paket **`java.lang`**, yang otomatis diimpor ke setiap program Java. Karena itu, semua kelas yang dibuat oleh programmer secara otomatis akan memiliki perilaku dan metode dasar yang berasal dari kelas `Object`. Beberapa metode utama yang disediakan antara lain:

1. toString() method

Metode `toString()` menyediakan representasi berupa `String` dari sebuah objek dan digunakan untuk mengonversi objek menjadi `String`. Secara default, metode `toString()` pada kelas `Object` akan mengembalikan sebuah string yang terdiri dari nama class tempat objek tersebut dibuat, karakter tanda `@`, dan representasi heksadesimal (tanpa tanda) dari nilai hash code objek tersebut.

```
public class Student {

    @Override
    public String toString() {
        return "Student object";
    }

    public static void main(String[] args) {
        Student s = new Student();          // membuat objek Student
        System.out.println(s.toString());   // memanggil toString()
    }
}
```

2. hashCode() method

Untuk setiap objek, JVM (Java Virtual Machine) menghasilkan sebuah angka unik yang disebut `hashcode`. Metode ini mengembalikan bilangan integer yang berbeda untuk setiap objek yang berbeda. Kegunaan Metode `hashCode()`:

- Metode ini mengembalikan nilai hash yang digunakan untuk mencari objek dalam sebuah koleksi.
- JVM menggunakan `hashCode()` saat menyimpan objek ke dalam struktur data berbasis hashing seperti `HashSet`, `HashMap`, dan `Hashtable`.
- Keuntungan utama menyimpan objek berdasarkan hash code adalah proses pencarian menjadi lebih cepat dan efisien.

```
public class Student {

    int roll;

    @Override
    public int hashCode() {
        return roll;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;

        Student s2 = new Student();
        s2.roll = 102;

        System.out.println("HashCode s1: " + s1.hashCode());
        System.out.println("HashCode s2: " + s2.hashCode());
    }
}
```

3. equals(object obj) method

Metode `equals()` digunakan untuk membandingkan objek yang diberikan dengan objek saat ini. Biasanya**,** disarankan untuk meng-override metode ini ketika kita ingin menentukan kondisi kesetaraan khusus untuk objek buatan sendiri.

```java
public class Student {

    int roll;

    @Override
    public boolean equals(Object o) {
        if (o instanceof Student) {
            return this.roll == ((Student) o).roll;
        }
        return false;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;

        Student s2 = new Student();
        s2.roll = 101;

        Student s3 = new Student();
        s3.roll = 102;

        System.out.println("s1.equals(s2): " + s1.equals(s2));
        System.out.println("s1.equals(s3): " + s1.equals(s3));
    }
}
```

4. finalize() method

Metode `finalize()` dipanggil tepat sebelum sebuah objek dikumpulkan oleh Garbage Collector. Metode ini dipanggil oleh Garbage Collector pada sebuah objek ketika ia menentukan bahwa tidak ada lagi referensi terhadap objek tersebut.Kita dapat meng-override metode `finalize()` untuk melepaskan resource sistem, melakukan aktivitas pembersihan (clean-up), dan meminimalkan kebocoran memori.

```
public class Geeks {
    public static void main(String[] args) {

        Geeks t = new Geeks();
        System.out.println(t.hashCode());

        t = null;

        System.gc();

        System.out.println("end");
    }

    @Override protected void finalize()
    {
        System.out.println("finalize method called");
    }
}
```

5. getClass() method

Metode `getClass()` mengembalikan class object dari objek `this` dan digunakan untuk mendapatkan kelas sebenarnya (runtime class) dari objek tersebut. Metode ini juga dapat digunakan untuk mendapatkan metadata dari kelas tersebut. Class object yang dikembalikan adalah objek yang dikunci (locked) oleh static synchronized methods dari kelas yang direpresentasikan. Karena metode ini bertipe final, maka **tidak dapat dioverride**.

```
public class Geeks {
    public static void main(String[] args)
    {
        Object o = new String("GeeksForGeeks");
        Class c = o.getClass();
        System.out.println("Class of Object o is: "
                           + c.getName());
    }
}
```

6. clone()method

Metode `clone()` membuat dan mengembalikan objek baru yang merupakan salinan dari objek saat ini.

```java
public class Book implements Cloneable {
    private String t;
    public Book(String t) {
        this.t = t;
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    public String getTitle() {
        return t;
    }

    public static void main(String[] args) {
        try {
            Book b1 = new Book("Java Basics");
            Book b2 = (Book) b1.clone();

            System.out.println("Judul buku b1: " + b1.getTitle());
            System.out.println("Judul buku b2: " + b2.getTitle());
            System.out.println("Apakah b1 dan b2 objek yang sama? " + (b1 == b2));

        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

7. wait(), notif(), notifAll() (Concurrency methods)

Metode `wait()`, `notify()`, dan `notifyAll()` adalah metode concurrency di Java yang digunakan untuk sinkronisasi antar *thread*, khususnya saat beberapa *thread* berbagi *resource* yang sama (misalnya variabel atau objek). Metode-metode ini adalah bagian dari class Object (bukan Thread), sehingga semua objek di Java punya metode ini secara bawaan. Tapi penggunaannya hanya bisa dilakukan di dalam blok synchronized atau metode synchronized, karena tujuannya adalah mengatur thread yang mengunci (lock) objek tersebut.

```java
public class Book implements Cloneable {
    private String t;
    private String a;
    private int y;

    public Book(String t, String a, int y)
    {
        this.t = t;
        this.a = a;
        this.y = y;
    }

    @Override public String toString()
    {
        return t + " by " + a + " (" + y + ")";
    }

    @Override public boolean equals(Object o)
    {
        if (o == null || !(o instanceof Book)) {
            return false;
        }
        Book other = (Book)o;
        return this.t.equals(other.getTitle())
            && this.a.equals(other.getAuthor())
            && this.y == other.getYear();
    }

    @Override public int hashCode()
    {
        int res = 17;
        res = 31 * res + t.hashCode();
        res = 31 * res + a.hashCode();
        res = 31 * res + y;
        return res;
    }

    @Override public Book clone()
    {
        try {
            return (Book)super.clone();
        }
        catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
```

```
    @Override protected void finalize() throws Throwable
    {
        System.out.println("Finalizing " + this);
    }

    public String getTitle() { return t; }
    public String getAuthor() { return a; }
    public int getYear() { return y; }

    public static void main(String[] args)
    {
        Book b1 = new Book(
            "The Hitchhiker's Guide to the Galaxy",
            "Douglas Adams", 1979);
        System.out.println(b1);

        Book b2 = b1.clone();
        System.out.println(b2);

        System.out.println("b1 equals b2: "
                           + b1.equals(b2));

        System.out.println("b1 hash code: "
                           + b1.hashCode());
        System.out.println("b2 hash code: "
                           + b2.hashCode());
        b1 = null;
        System.gc();
    }
}
```

Constructs

Dalam Java, constructor memiliki peran penting dalam pembuatan objek. Constructor adalah blok kode khusus yang dipanggil saat sebuah objek dibuat. Tugas utamanya adalah menginisialisasi objek, menyiapkan keadaan internalnya, atau memberikan nilai awal (default) pada atribut-atributnya. Proses ini terjadi secara otomatis ketika kita menggunakan keyword `new` untuk membuat sebuah objek.

1. Default Constructor

Constructor yang tidak memiliki parameter disebut default constructor. Default constructor bersifat tidak terlihat (*invisible*). Jika kita menulis constructor tanpa argumen, compiler tidak akan membuat default constructor secara otomatis. Default constructor bisa bersifat implicit default constructor dan explicit default constructor.

```java
class Geeks{

    // Default Constructor
    Geeks() {
        System.out.println("Default constructor");

    }

    public static void main(String[] args)
    {
        Geeks hello = new Geeks();
    }
}
```

2. Parameterized Constructor

Constructor yang memiliki parameter disebut parameterized constructor. Jika kita ingin menginisialisasi field (atribut) dari kelas dengan nilai kita sendiri, maka kita menggunakan parameterized constructor.

```java
class Geeks {

    // data members of the class
    String name;
    int id;

    Geeks(String name, int id) {
        this.name = name;
        this.id = id;
    }
}

public class Main
{
    public static void main(String[] args)
    {
        // This would invoke the parameterized constructor
        Geeks geek1 = new Geeks("Sweta", 68);
        System.out.println("GeekName: " + geek1.name
                           + " and GeekId: " + geek1.id);
    }
}
```

3. Copy Constructor

Copy constructor menerima objek lain sebagai parameter, yang kemudian menyalin data dari objek yang diterima ke objek baru yang dibuat.

```
class Geeks {

    String name;
    int id;

    // Parameterized Constructor
    Geeks(String name, int id)
    {
        this.name = name;
        this.id = id;
    }

    // Copy Constructor
    Geeks(Geeks obj2)
    {
        this.name = obj2.name;
        this.id = obj2.id;
    }
}

public class Main {
    public static void main(String[] args)
    {
        System.out.println("First Object");
        Geeks geek1 = new Geeks("Sweta", 68);
        System.out.println("GeekName: " + geek1.name
                           + " and GeekId: " + geek1.id);

        System.out.println();

        Geeks geek2 = new Geeks(geek1);
        System.out.println(
            "Copy Constructor used Second Object");
        System.out.println("GeekName: " + geek2.name
                           + " and GeekId: " + geek2.id);
    }
}
```

Constructor Overloading

Ini adalah konsep penting dalam OOP (Object-Oriented Programming) yang berkaitan dengan constructor, yaitu constructor overloading. Konsep ini memungkinkan kita untuk membuat beberapa constructor dalam satu kelas dengan daftar parameter yang berbeda-beda.

```java
class Geeks {
    // constructor with one argument
    Geeks(String name)
    {
        System.out.println("Constructor with one "
                           + "argument - String: " + name);
    }
    // constructor with two arguments
    Geeks(String name, int age)
    {
        System.out.println(
            "Constructor with two arguments: "
            + " String and Integer: " + name + " " + age);
    }
    // Constructor with one argument but with different
    Geeks(long id)
    {
        System.out.println(
            "Constructor with one argument: "
            + "Long: " + id);
    }
}

public class Main {
    public static void main(String[] args)
    {

        Geeks geek2 = new Geeks("Sweta");

        Geeks geek3 = new Geeks("Amiya", 28);

        Geeks geek4 = new Geeks(325614567);
    }
}
```

Passing Object in Method

Di java kita bisa meneruskan objek ke metode yang dapat kita lihat pada contoh dibawah.

```java
class ObjectPassDemo {
    int a, b;

    ObjectPassDemo(int i, int j)
    {
        a = i;
        b = j;
    }

    // Method
    boolean equalTo(ObjectPassDemo o)
    {
        return (o.a == a && o.b == b);
    }
}

public class Main {

    public static void main(String args[]) {
        ObjectPassDemo ob1 = new ObjectPassDemo(100, 22);
        ObjectPassDemo ob2 = new ObjectPassDemo(100, 22);
        ObjectPassDemo ob3 = new ObjectPassDemo(-1, -1);

        System.out.println("ob1 == ob2: "
                + ob1.equalTo(ob2));
        System.out.println("ob1 == ob3: "
                + ob1.equalTo(ob3));
    }
}
```

Getter dan Setter

Getter dan Setter adalah metode yang digunakan untuk melindungi data dan membuat kode Anda lebih aman. Getter dan Setter memudahkan programmer dalam mengatur (set) dan mengambil (get) nilai dari suatu tipe data tertentu. Getter mengembalikan nilai (accessor), yaitu mengembalikan nilai dari tipe data seperti int, String, double, float, dan lain-lain. Demi kemudahan dalam pemrograman, metode getter diawali dengan kata "get" yang diikuti dengan nama variabel. Setter digunakan untuk mengatur atau memperbarui nilai (mutator). Setter menetapkan nilai untuk variabel apa pun yang digunakan dalam suatu kelas. Metode setter diawali dengan kata "set" yang diikuti dengan nama variabel.

Contoh 1

```java
class GetSet {

    private String name;

    // Method 1 - Getter
    public String getName() { return name; }

    // Method 2 - Setter
    public void setName(String N)
    {

        this.name = N;
    }
}


class GFG {

    public static void main(String[] args)
    {
        GetSet obj = new GetSet();

        obj.setName("Geeks for Geeks");
        System.out.println(obj.getName());
    }
}
```

Contoh 2

```java
class GetSet {
    private int num;
    // Method 1 - Setter
    public void setNumber(int number)
    {
        if (number < 1 || number > 10) {
            throw new IllegalArgumentException();
        }
        num = number;
    }
    // Method 2 - Getter
    public int getNumber() { return num; }
}

class GFG {
    public static void main(String[] args)
    {
        GetSet obj = new GetSet();
        obj.setNumber(5);
        System.out.println(obj.getNumber());
    }
}
```

Inheritance

Inheritance atau pewarisan adalah salah satu konsep utama dalam pemrograman berorientasi objek (Object-Oriented Programming/OOP) yang memungkinkan sebuah kelas mewarisi sifat (atribut) dan perilaku (metode) dari kelas lain. Dalam Java, inheritance digunakan untuk menciptakan hubungan **"**IS-A**"** antara dua kelas, yaitu kelas anak (*subclass*) dan kelas induk (*superclass*). Subclass dapat menggunakan kembali kode yang telah ditulis di superclass, sehingga mengurangi duplikasi kode dan memudahkan pemeliharaan program. Melalui inheritance, subclass tidak hanya dapat menggunakan anggota yang dimiliki superclass, tetapi juga dapat menambahkan anggota baru atau memodifikasi perilaku yang ada melalui mekanisme overriding. Terdapat beberapa type inheritance di java yaitu sebagai berikut.

1. Singel Inheritance

Single inheritance adalah bentuk pewarisan paling sederhana di mana sebuah class anak (*child class*) hanya mewarisi dari satu class induk (*parent class*). Dalam konsep ini, *child class* dapat

menggunakan semua properti dan method yang dimiliki *parent class*, sekaligus menambahkan fitur miliknya sendiri. Kita dapat melihat pada contoh code berikut.

```
class Vehicle { //Super class
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

public class Test {
    public static void main(String[] args) {
        Car obj = new Car();
    }
}
```

2. Multilevel Inheritance

Multilevel inheritance terjadi ketika pewarisan dilakukan secara bertingkat, di mana sebuah *class* mewarisi dari *class* lain, dan *class* berikutnya mewarisi lagi dari *class* tersebut. Kita dapat melihat pada contoh code berikut.

```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
class FourWheeler extends Vehicle {
    FourWheeler() {
        System.out.println("4 Wheeler Vehicles");
    }
}
class Car extends FourWheeler {
    Car() {
        System.out.println("This 4 Wheeler Vehicle is a Car");
    }
}
```

```java
public class Geeks {
    public static void main(String[] args) {
        Car obj = new Car();
    }
}
```

3. Hierarchical Inheritance

Hierarchical inheritance digunakan saat satu class induk memiliki banyak class anak. Semua class anak mewarisi atribut dan method yang sama dari class induk, sehingga kode yang umum hanya ditulis sekali di parent class. Kita dapat melihat pada contoh code berikut.

```java
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

class Bus extends Vehicle {
    Bus() {
        System.out.println("This Vehicle is Bus");
    }
}

public class Test {
    public static void main(String[] args) {
        Car obj1 = new Car();
        Bus obj2 = new Bus();
    }
}
```

4. Multiple Inheritance

Multiple inheritance tidak bisa dilakukan secara langsung antar class di Java karena bisa menimbulkan konflik pewarisan, tetapi bisa dilakukan lewat *interface*. Dengan *interface*, satu class

dapat mengimplementasikan banyak interface sekaligus dan mewarisi perilaku dari semuanya. Kita dapat melihat pada contoh code berikut.

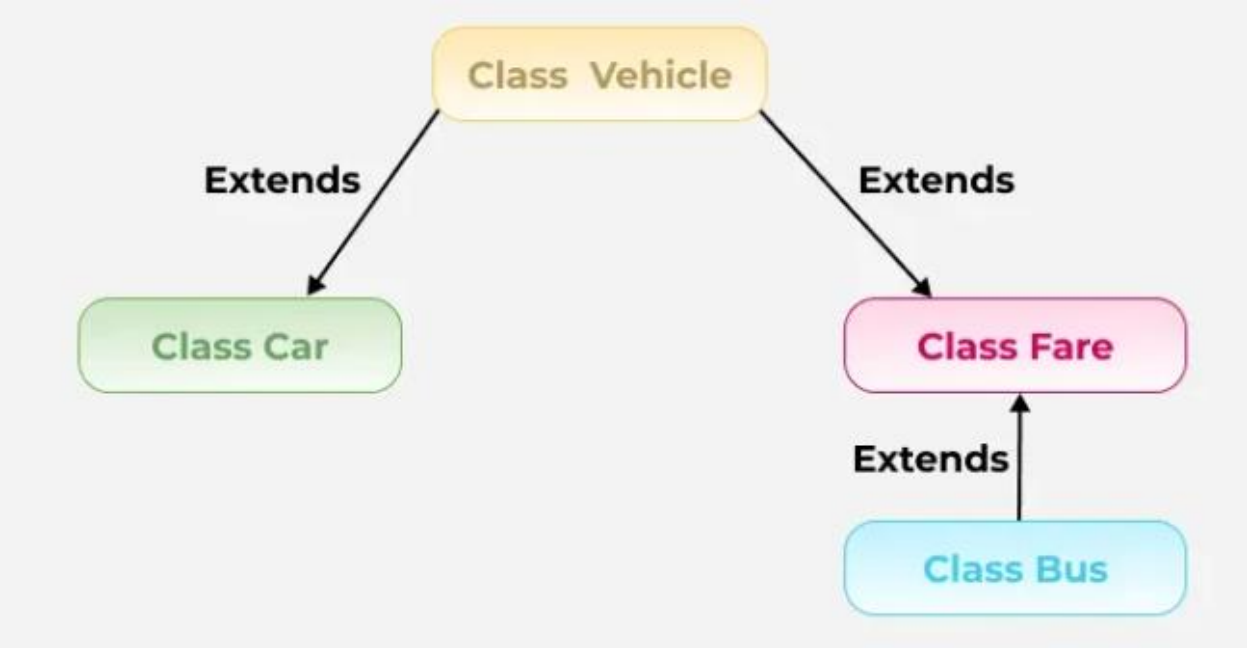
```
interface LandVehicle {
    default void landInfo() {
        System.out.println("This is a LandVehicle");
    }
}
interface WaterVehicle {
    default void waterInfo() {
        System.out.println("This is a WaterVehicle");
    }
}


class AmphibiousVehicle implements LandVehicle, WaterVehicle {
    AmphibiousVehicle() {
        System.out.println("This is an AmphibiousVehicle");
    }
}

public class Test {
    public static void main(String[] args) {
        AmphibiousVehicle obj = new AmphibiousVehicle();
        obj.waterInfo();
        obj.landInfo();
    }
}
```

5. Hybrid Inheritance

Hybrid inheritance merupakan kombinasi dari beberapa jenis inheritance sekaligus, misalnya gabungan hierarchical dan multiple inheritance. Karena Java tidak mendukung multiple inheritance secara langsung, hybrid inheritance di Java biasanya dilakukan dengan menggabungkan pewarisan antar class dengan implementasi beberapa interface.



TUGAS

- Kumpulkan dalam bentuk folder zip dengan nama: **`PBO_W4S2_NIM.zip`**
- Isi folder: semua file `.java`
- **Berikan komentar pada setiap program** untuk menjelaskan baris code atau blok logika atau penjelasan fungsi program.